

(Somaiya Vidyavihar University)

Department of Computer Engineering

Batch: C3 Roll No.: 16010123325

Experiment / Assignment / Tutorial No: 07

Title: Implementation of CNN Deep Learning Architecture

Aim: To understand & implement CNN using python.

Expected Outcome of Experiment:

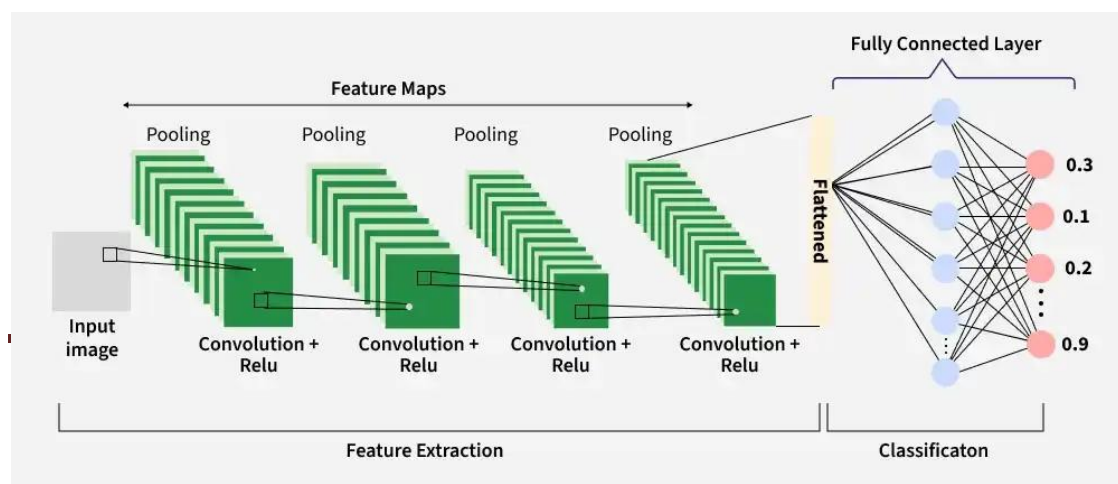
CO4:

Books/ Journals/ Websites referred:

1. “Deep Learning Core Concepts, Methods and Applications”, M. Gopal, Pearson, 2022.
2. <https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/>
3. <https://medium.com/>

Theory of Convolutional Neural Networks (CNNs):

Convolutional Neural Networks (CNNs), also known as ConvNets, are neural network architectures inspired by the human visual system and are widely used in computer vision tasks. They are designed to process structured grid-like data, especially images by capturing spatial relationships between pixels. CNN for classification task has two kinds of layers; one deals with feature engineering, and other performs classification. The feature engineering layers are called convolution layers and the classification layers are called convolutional layers.



Department of Computer Engineering

Key Components of CNN

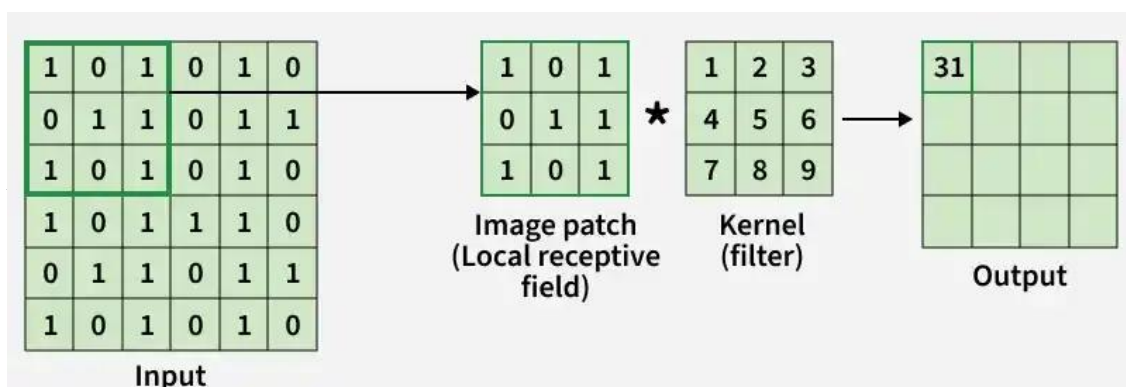
A complete Convolution Neural Networks architecture also known as convnets is a sequence of layers and every layer transforms one volume to another through a differentiable function.

1. Input Layer

The input layer receives the raw image data and passes it to the network for processing. In CNNs, input is typically a 3D volume (width $\times$ height $\times$ depth). This layer stores pixel values of the image (e.g., $32 \times 32 \times 3$ for RGB images); preserves the spatial structure of the image for further feature extraction.

2. Convolutional Layers

Convolution layers are key building blocks of convolutional neural networks (CNNs) that are responsible for extracting important features from the input data. They apply convolution operation to the input data which involves a set of learnable filters (kernels) that slide over the image and compute the dot product between the filter weights and corresponding image patches, producing feature maps. Uses small filters (e.g., 2×2 , 3×3 , 5×5) to scan the input image. This process allows the network to detect patterns such as edges, textures and shapes in the input images. E.g. Using 12 filters results in an output volume of $32 \times 32 \times 12$.



Padding: Zeros or other values may be added around the input to control the spatial dimensions of the output.

Common types: "valid" (no padding) and "same" (pads output so feature map dimensions match input).

(Somaiya Vidyavihar University)

Department of Computer Engineering

Mathematical Representation:

For 2-D input image, using 2-D kernel (filter) the convolution operation can be represented in the following form:

$$J(x, y) = I * K(\mathbf{x}, \mathbf{y}) = \sum_i \sum_j I(i, j) K(x-i, y-j)$$

where, I represents 2-D input image; K is the 2-D kernel/filter

(Somaiya Vidyavihar University)

Department of Computer Engineering

The feature maps in first convolution layer C^1 can be mathematically written as:

$$C^1_j = \sigma ((X * K^1_{i,j}) + b^1_j) = \sigma (a_j)$$

where,

$K^1_{i,j}$ denotes elements of first layer kernel

(i, j) indices denotes the indices of the current and next feature map (here $i = 1$)

$\sigma(.)$ us the non linear activation function, and a_j is the activation value; $j= 1, 2$

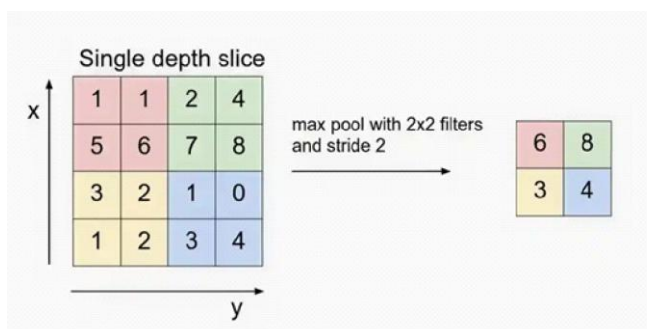
The operation $*$ denotes 2 D convolution.

3. Activation Layer:

The Activation Layer introduces non-linearity into the network by applying an element-wise activation function to the output of the convolution layer. After convolution, a non-linear function is often applied allowing the network to learn complex relationships in data beyond linear relationships. The output dimensions remain unchanged (e.g., $32 \times 32 \times 12$). Common activation functions: ReLU, Sigmoid, tanh, Leaky ReLU etc.

4. Pooling Layer

Pooling (Optional): The Pooling Layer is used to reduce the spatial dimensions of the feature maps and retain the most important information. This makes computation faster, reducing memory usage and helping to prevent overfitting. It is typically inserted between convolutional layers in a CNN. Pooling reduces width and height while keeping depth unchanged. Common types include Max Pooling and Average Pooling. Example: Using 2×2 max pooling with stride 2 reduces the volume from $32 \times 32 \times 12$ to $16 \times 16 \times 12$.



(Somaiya Vidyavihar University)

Department of Computer Engineering

5. Flattening

Flattening converts the multi-dimensional feature maps into a one-dimensional vector after convolution and pooling. This vector is then passed to the fully connected layer for classification or regression.

$$z = \text{flatten} \{ S_j^2 \mid j = 1, 2, 3, 4 \}$$

Example: Flattening $16 \times 16 \times 12$ results in a vector of size 3072 ($16 \times 16 \times 12$).

6. Fully Connected Layer

The fully connected (dense) layer performs high-level reasoning using extracted features and produces the final classification scores.

Example: The 3072-length vector is connected to neurons for classification

7. Output Layer

The output layer converts final scores into probabilities using activation functions like Sigmoid (binary classification) or Softmax (multi-class classification).

Example: For 10 classes, Softmax produces 10 probability values each representing the likelihood of a class.

Implementation Steps to demonstrate Convolutional Neural Network (illustrating how each layer processes and transforms the input image. [Refer](#))

Step 1: Import Required Libraries: Import TensorFlow for CNN operations and Matplotlib for visualization.

Step 2: Load and Preprocess the Image: Load the image convert it to grayscale, resize it to 300×300 and normalize pixel values.

Department of Computer Engineering

Step 3: Define Convolution Kernel: define an edge detection filter (Laplacian kernel) to extract important image features.

Step 4: Apply Convolution Layer-The convolution layer applies the filter to the image to detect edges and features.

Step 5: Apply Activation Function- e.g. ReLU removes negative values and introduces non-linearity into the network.

Step 6: Apply Max Pooling Layer- Max pooling reduces spatial dimensions while keeping important features

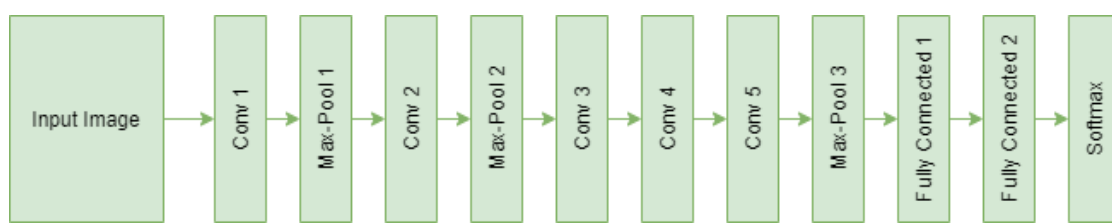
Step 7: Apply Flatten Layer- The flatten layer converts 2D feature maps into a 1D feature vector for fully connected layers.

Step 8: Add Fully Connected (Dense) Layer- The fully connected layer learns high-level patterns from the flattened feature vector and produces output predictions.

Different Types of CNN Models

1. LeNet: LeNet developed by Yann LeCun and his colleagues in the late 1990s was one of the first successful CNNs designed for handwritten digit recognition. It laid the foundation for modern CNNs and achieved high accuracy on the MNIST dataset which contains 70,000 images of handwritten digits (0-9).

2. AlexNet: AlexNet is a CNN architecture that was developed by Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton in 2012. It was the first CNN to win the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) a major image recognition competition. It consists of several layers of convolutional and pooling layers followed by fully connected layers. The architecture includes five convolutional layers, three pooling layers and three fully connected layers.



4. GoogleNet: GoogleNet also known as InceptionNet is renowned for achieving high accuracy in image classification while using fewer parameters and computational resources compared to

(Somaiya Vidyavihar University)

Department of Computer Engineering

other state-of-the-art CNNs. The core component of GoogleNet allows the network to learn features at different scales simultaneously to enhance performance.

5. VGG: VGGs are developed by the Visual Geometry Group at Oxford, it uses small 3x3 convolutional filters stacked in multiple layers, creating a deep and uniform structure. Popular variants like VGG-16 and VGG-19 achieved state-of-the-art performance on the ImageNet dataset demonstrating the power of depth in CNNs.

Training a Convolutional Neural Network

CNNs are trained using a supervised learning approach. This means that the CNN is given a set of labeled training images. The CNN learns to map the input images to their correct labels.

The training process for a CNN involves the following steps:

1. **Data Preparation:** The training images are preprocessed to ensure that they are all in the same format and size.
2. **Loss Function:** A loss function is used to measure how well the CNN is performing on the training data. The loss function is typically calculated by taking the difference between the predicted labels and the actual labels of the training images.
3. **Optimizer:** An optimizer is used to update the weights of the CNN in order to minimize the loss function.
4. **Backpropagation:** Backpropagation is a technique used to calculate the gradients of the loss function with respect to the weights of the CNN. The gradients are then used to update the weights of the CNN using the optimizer.

Evaluation Metrics

Efficiency of CNN can be evaluated using a variety of criteria. Among the most popular metrics are:

Accuracy: Accuracy is the percentage of test images that the CNN correctly classifies.

Precision: Precision is the percentage of test images that the CNN predicts as a particular class and that are actually of that class.

Recall: Recall is the percentage of test images that are of a particular class and that the CNN predicts as that class.

F1 Score: The F1 Score is a harmonic mean of precision and recall. It is a good metric for evaluating the performance of a CNN on classes that are imbalanced.

(Somaiya Vidyavihar University)

Department of Computer Engineering

Problem Definition:

To implement CNN AlexNet architecture for object classification.
(<https://www.geeksforgeeks.org/machine-learning/ml-getting-started-with-alexnet/>)

Details of data set used: *CIFAR-10 Dataset*

Implementation Details:

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Load dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Normalize (0-255 → 0-1)
x_train, x_test = x_train / 255.0, x_test / 255.0

print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 4s 0us/step
Training shape: (50000, 32, 32, 3)
Testing shape: (10000, 32, 32, 3)

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(8,8))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i][0]])
    plt.axis('off')
plt.show()
```


(Somaiya Vidyavihar University)

Department of Computer Engineering

frog



truck



truck



deer



automobile



automobile



bird



horse



ship



```
model = models.Sequential()

# Conv Layer 1
model.add(layers.Conv2D(64, (3,3), activation='relu', input_shape=(32,32,3)))
model.add(layers.MaxPooling2D((2,2)))

# Conv Layer 2
model.add(layers.Conv2D(128, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

# Conv Layer 3
model.add(layers.Conv2D(256, (3,3), activation='relu'))

# Flatten
model.add(layers.Flatten())

# Fully Connected Layers
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(128, activation='relu'))

# Output Layer (Softmax for 10 classes)
model.add(layers.Dense(10, activation='softmax'))

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, pr
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

(Somaiya Vidyavihar University)

Department of Computer Engineering

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

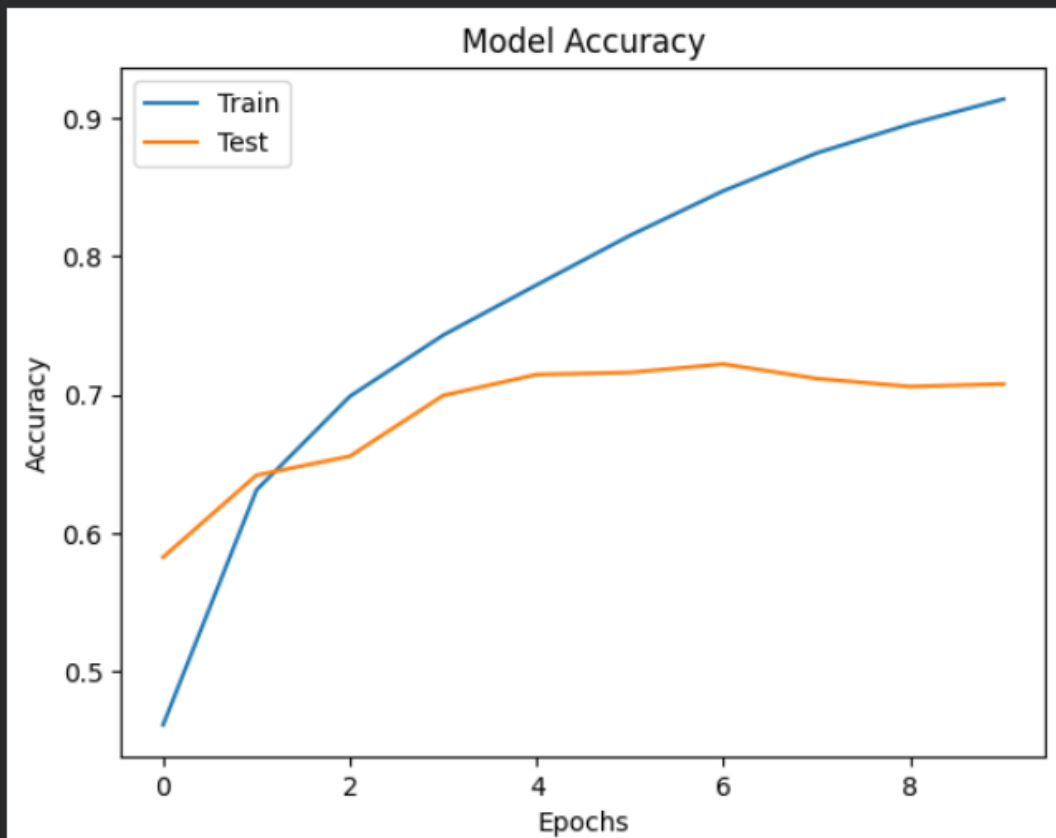
history = model.fit(x_train, y_train,
                   epochs=10,
                   validation_data=(x_test, y_test))
```

Epoch	Time	Step	Accuracy	Loss	Val Accuracy	Val Loss
Epoch 1/10	235s	149ms/step	0.4611	1.4750	0.5824	1.1737
Epoch 2/10	225s	144ms/step	0.6310	1.0539	0.6417	1.0038
Epoch 3/10	226s	145ms/step	0.6987	0.8612	0.6554	1.0003
Epoch 4/10	224s	143ms/step	0.7430	0.7305	0.6994	0.8816
Epoch 5/10	261s	143ms/step	0.7793	0.6221	0.7145	0.8628
Epoch 6/10	222s	142ms/step	0.8151	0.5231	0.7160	0.9102
Epoch 7/10	222s	142ms/step	0.8474	0.4324	0.7221	0.9099
Epoch 8/10	221s	141ms/step	0.8747	0.3540	0.7116	0.9861
Epoch 9/10	265s	143ms/step	0.8957	0.2916	0.7058	1.1212
Epoch 10/10	221s	142ms/step	0.9138	0.2418	0.7077	1.2125

(Somaiya Vidyavihar University)

Department of Computer Engineering

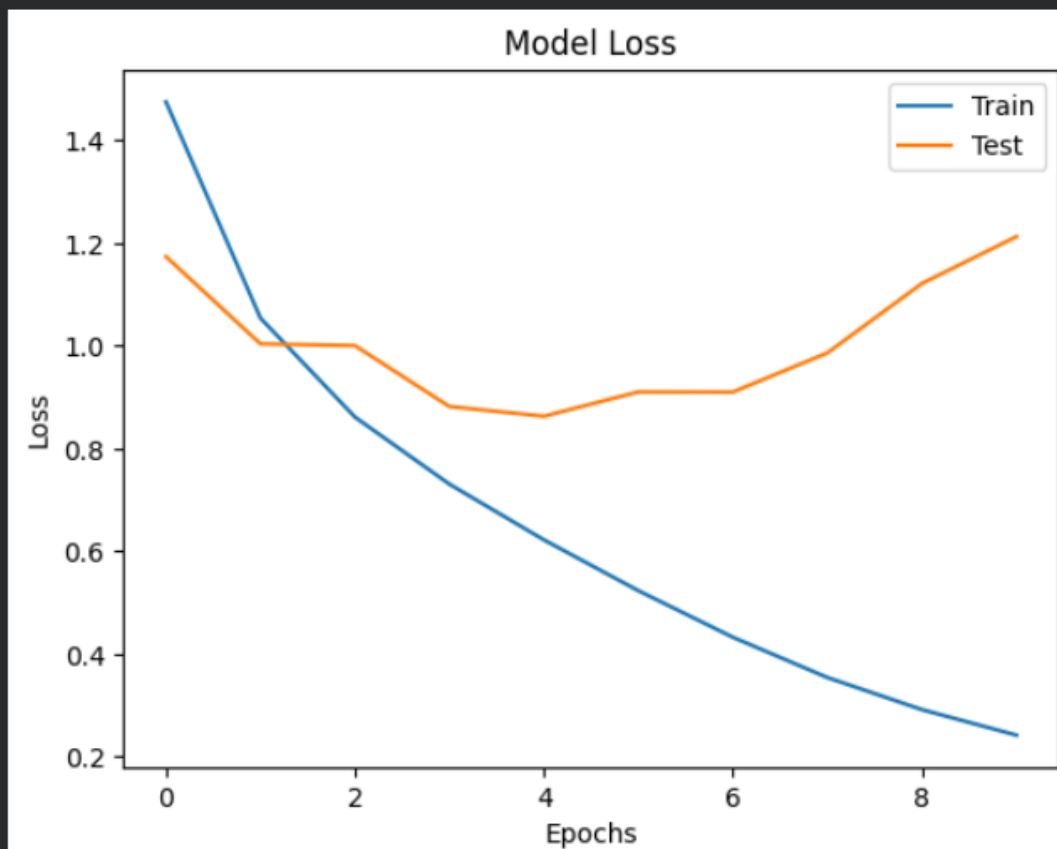
```
plt.plot(history.history['accuracy'])  
plt.plot(history.history['val_accuracy'])  
plt.title('Model Accuracy')  
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.legend(['Train', 'Test'])  
plt.show()
```



(Somaiya Vidyavihar University)

Department of Computer Engineering

```
plt.plot(history.history['loss'])  
plt.plot(history.history['val_loss'])  
plt.title('Model Loss')  
plt.xlabel('Epochs')  
plt.ylabel('Loss')  
plt.legend(['Train', 'Test'])  
plt.show()
```



(Somaiya Vidyavihar University)

Department of Computer Engineering

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

```
313/313 ————— 10s 31ms/step - accuracy: 0.7077 - loss: 1.2125
Test Accuracy: 0.7077000141143799
```

```
import numpy as np

predictions = model.predict(x_test)

plt.figure(figsize=(8,8))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_test[i])
    plt.title(class_names[np.argmax(predictions[i])])
    plt.axis('off')
plt.show()
```

```
313/313 ————— 11s 33ms/step
```

(Somaiya Vidyavihar University)

Department of Computer Engineering

cat



ship



automobile



airplane



deer



frog



cat



frog



cat



Conclusion:

The CNN model based on AlexNet architecture was successfully implemented on the CIFAR-10 dataset. The model effectively learned features using convolution and pooling layers and achieved good classification accuracy. Training and validation plots showed proper learning behavior, demonstrating the effectiveness of deep learning in image classification tasks.

(Somaiya Vidyavihar University)

Department of Computer Engineering

Post Lab :

1. Explain what is the significance of splitting the data into training and validation and number of epochs in the training of CNN model.

Data Splitting (Training vs Validation)

- **Training Data**
 - Used to train the CNN model
 - Model learns patterns and features from this data
- **Validation Data**
 - Used to evaluate model performance during training
 - Helps detect **overfitting** (model memorizing instead of learning)

Why it is important:

- Ensures model generalizes well to unseen data
- Helps tune hyperparameters (learning rate, layers, etc.)
- Prevents overfitting and underfitting

Number of Epochs

- An **epoch** = one complete pass over the training dataset

Effect of epochs:

- Too few epochs → **Underfitting** (model not learned properly)
- Too many epochs → **Overfitting** (memorizes data)
- Optimal epochs → best validation accuracy

(Somaiya Vidyavihar University)

Department of Computer Engineering

Splitting data ensures proper training and evaluation, while the number of epochs controls how well the model learns without overfitting or underfitting.

2. Explain any one light weight CNN model suitable for mobile based application.

MobileNet

MobileNet is a lightweight CNN architecture designed for mobile and embedded devices.

Key Features:

- Uses **Depthwise Separable Convolutions**
 - Reduces computation significantly
- Smaller model size
- Faster execution
- Lower power consumption

Why suitable for mobile:

- Efficient memory usage
- Real-time performance
- Maintains good accuracy with fewer parameters

Applications:

- Face detection
- Object detection
- Image classification in mobile apps

(Somaiya Vidyavihar University)

Department of Computer Engineering

MobileNet is suitable for mobile applications due to its efficient architecture, reduced computation, and ability to deliver good accuracy with low resource usage.